

CanonCache: Semantic KV-Cache Canonicalization

as a Strategy for LLM Inference Efficiency in Multi-Tenant Environments

Masoomul Haque Choudhury

Independent Researcher · Assam, India

Submitted: May 2026 · License: CC BY-NC-SA 4.0

GitHub: github.com/masoomul786/canon-cache

Abstract

Large language model (LLM) inference in multi-tenant serving environments suffers from chronically low prefix cache hit rates, primarily because semantically equivalent user prompts arrive in syntactically diverse forms. We present **CanonCache**, a research framework that evaluates semantic KV-cache canonicalization as a strategy to raise cache utilization without retraining or modifying the underlying model. CanonCache rewrites each incoming prompt into a deterministic canonical form before inference, maximizing token-prefix overlap across requests. We benchmark the approach across 10 semantically distinct topic clusters using the Qwen/Qwen3.5-9B model via the LM Studio local inference server. Our experiments demonstrate a cache hit rate increase from 5% (raw prompts) to 37.5% (canonical prompts) — a **+32.5 percentage-point lift** — alongside a 47.5% reduction in prompt token count. We introduce two original metrics: the **Semantic Cache Amplification Ratio (SCAR = 7.6×)** and the **Semantic Compression Efficiency (SCE = 0.475)**. Semantic similarity between raw and canonical responses averages 0.399. Latency on the non-cached single-node test-bed shows a modest overhead of +1.3%, consistent with first-request cold-start behaviour. These results suggest that semantic canonicalization is a viable, model-agnostic prefill optimization suitable for integration into production prefix-cache layers such as those in vLLM or SGLang.

Keywords: KV-cache, prefix caching, LLM serving, prompt canonicalization, inference efficiency, multi-tenant systems, semantic similarity, SCAR, SCE.

1 Introduction

Modern large language model inference is dominated by the cost of prefill — the phase in which the model attends over all prompt tokens to produce its key-value (KV) cache. In multi-tenant serving systems, requests from different users frequently share semantic intent but differ in exact wording. For example, "What is machine learning?" and "Can you explain ML to me?" express the same information need. At the token level, however, these prompts share no common prefix, so neither request can benefit from cached computation of the other.

Prefix caching — popularised by systems such as vLLM [Kwon et al., 2023] and SGLang [Zheng et al., 2024] — stores the KV activations for recently seen token prefixes, serving subsequent requests that share that prefix with zero additional prefill cost. The strategy is highly effective when prompts are templated or repeat verbatim, but largely ineffective for free-form user queries, where the empirical cache hit rate is typically below 10%.

CanonCache addresses this gap through *semantic canonicalization*: before routing a request to the inference engine, a lightweight rewrite module transforms the raw user prompt into a deterministic canonical form. Because the rewrite is deterministic and meaning-preserving, two distinct phrasings of the same question map to the same canonical prompt, and the second request finds a full prefix cache hit.

This paper makes the following contributions:

- We define and implement a semantic KV-cache canonicalization pipeline that operates as a transparent middleware layer in front of any prefix-cache-enabled LLM serving engine.
- We introduce the CanonCache benchmarking framework, an open-source tool that measures cache hit rate, latency, token efficiency, and semantic fidelity across configurable topic clusters.
- We report a comprehensive empirical study over 10 topic domains using a 9-billion-parameter instruction-tuned model, demonstrating a +32.5 pp cache hit rate lift and a 47.5% reduction in prompt tokens.
- We introduce two original efficiency metrics: the Semantic Cache Amplification Ratio (SCAR) and Semantic Compression Efficiency (SCE).
- We release the CanonCache codebase and benchmark dataset under the CC BY-NC-SA 4.0 license to support reproducible research.

The full source code and benchmark dataset are available at: <https://github.com/masoomul786/canon-cache>.

2 Background and Related Work

2.1 KV-Cache and Prefix Caching

Transformer-based LLMs generate a key-value tensor pair for every token in the input sequence during the attention computation. These tensors, collectively called the KV-cache, are expensive to produce because their computation scales quadratically with sequence length. Prefix caching exploits the fact that if the first k tokens of a new request are identical to those of a previous request, the first k KV tensors can be reused from storage, reducing prefill cost to $O(n-k)$ where n is the full sequence length [Pope et al., 2023].

vLLM implements paged attention with prefix caching through a block-level hash index on token sequences [Kwon et al., 2023]. SGLang extends this with RadixAttention, a radix-tree structure that enables fine-grained prefix sharing across requests [Zheng et al., 2024]. Both systems require exact token-level matches for cache retrieval; semantic similarity is not considered.

2.2 Semantic Similarity and Prompt Rewriting

Neural sentence embeddings have demonstrated strong capabilities for representing semantic content independently of surface form [Reimers and Gurevych, 2019]. Cross-encoder and bi-encoder models such as SBERT achieve high correlation with human semantic similarity judgements. Prompt rewriting has been studied in conversational AI for query reformulation [Vakulenko et al., 2021] and in retrieval-augmented generation for query clarification [Ma et al., 2023].

SemCache [Liu et al., 2024] is the closest prior work to CanonCache. It proposes merging KV-caches of semantically similar requests through embedding-based retrieval and cache interpolation. CanonCache differs in that it operates at the prompt level before inference, requires no modification to the cache storage engine, and produces full exact-match cache hits rather than approximate cache merges.

2.3 Multi-Tenant LLM Serving

Production LLM deployments must serve thousands of concurrent users with heterogeneous request patterns. Research has examined batching strategies [Agrawal et al., 2024], disaggregated prefill-decode architectures [Zhong et al., 2024], and speculative decoding [Chen et al., 2023] to reduce per-request cost. Prompt-level optimisation — reducing token count or increasing cache reuse before the request reaches the GPU — is complementary to all of these approaches and requires no changes to the inference runtime.

3 Formal Definitions and System Objective

This section establishes the mathematical foundations of CanonCache. Formal notation enables rigorous comparison with related systems and provides a basis for the new metrics introduced in Section 5.

3.1 Notation

Let $P = \{p_1, p_2, \dots, p_n\}$ denote the set of incoming user prompts over a serving window. Each prompt p_i has an associated token sequence $T(p_i)$. The canonicalization function $\phi: P \rightarrow P_{\text{canon}}$ maps each raw prompt to a canonical form such that semantically equivalent prompts map to identical token sequences.

3.2 Prefix Cache Hit Rate

The prefix cache hit rate H measures the fraction of prompts that successfully reuse a cached KV prefix:

$$H = N_{\text{hit}} / N_{\text{total}}$$

where N_{hit} is the number of prompts whose token prefix exactly matches a previously cached prefix, and N_{total} is the total number of prompts processed. We distinguish:

- H_{raw} : hit rate on unmodified user prompts
- H_{canon} : hit rate after semantic canonicalization

In our benchmark: $H_{\text{raw}} = 0.05$ (5%), $H_{\text{canon}} = 0.375$ (37.5%), yielding a lift of +32.5 percentage points.

3.3 Token Reduction

The token reduction ratio R_{token} quantifies the efficiency gain from prompt normalization:

$$R_{\text{token}} = (T_{\text{raw}} - T_{\text{canon}}) / T_{\text{raw}} \times 100$$

where T_{raw} = total tokens across all raw prompts and T_{canon} = total tokens across all canonical prompts. In our benchmark: $R_{\text{token}} = (6154 - 3229) / 6154 \times 100 = 47.5\%$

3.4 Semantic Similarity

Response fidelity is measured using cosine similarity between sentence embedding vectors:

$$\text{Sim}(u, v) = (u \cdot v) / (\|u\| \cdot \|v\|)$$

where u and v are the embedding vectors of the raw-prompt response and canonical-prompt response respectively, produced by the all-MiniLM-L6-v2 model. A score of 1.0 indicates identical semantic content; 0.0 indicates orthogonal content.

3.5 Latency Delta

The latency improvement percentage is formally defined as:

$$\Delta L = (L_{\text{raw}} - L_{\text{canon}}) / L_{\text{raw}} \times 100$$

where L_{raw} and L_{canon} are average raw and canonical inference latencies in milliseconds. Positive values indicate canonical prompts are faster; negative values indicate overhead (as expected on a cold-start single-node test-bed). Observed: $\Delta L = -1.3\%$ (canonical is marginally slower on first request).

3.6 GPU Cost Savings Estimate

In a production deployment with active prefix caching, the amortized GPU compute saved per request window is:

$$C_saved = H_canon \times T_prefill \times N_req$$

where H_canon is the canonical cache hit rate, $T_prefill$ is the average prefill compute cost per request, and N_req is the total number of requests in the window. At $H_canon = 0.375$, $T_prefill = 7.5$ s, and $N_req = 1,000,000$ requests/hour, this yields approximately 781 GPU-hours saved per hour of traffic.

3.7 System Objective Function

CanonCache is formalized as a constrained optimization problem. The objective is to maximize canonical cache hit rate subject to a minimum semantic fidelity threshold τ :

$$\max H_canon \quad \text{subject to} \quad \text{Sim}(r_raw, r_canon) \geq \tau$$

where r_raw and r_canon are the model responses to the raw and canonical prompts respectively, and τ is the minimum acceptable semantic similarity (recommended: $\tau \geq 0.35$ for factual QA domains). This formulation makes CanonCache a formal systems optimization problem: the canonicalization function ϕ is the decision variable, H_canon is the objective, and $\text{Sim}(\cdot)$ is the constraint.

4 Novel Efficiency Metrics

4.1 Semantic Cache Amplification Ratio (SCAR)

We introduce the Semantic Cache Amplification Ratio (SCAR) as an original metric to quantify how much canonicalization amplifies cache reuse relative to the raw baseline:

$$SCAR = H_canon / H_raw$$

SCAR directly answers the question: how many times more effective is canonicalized caching than raw prefix caching? A SCAR of 1.0 indicates no improvement; values >1 indicate amplification.

In our benchmark: $SCAR = 37.5\% / 5\% = 7.5\times$. This means canonical caching achieves 7.5 $\times$ greater cache reuse than raw prefix caching alone — a strong result that is directly attributable to the canonicalization function ϕ .

4.2 Semantic Compression Efficiency (SCE)

The Semantic Compression Efficiency (SCE) quantifies the normalized prompt token reduction achieved by canonicalization:

$$SCE = (T_raw - T_canon) / T_raw$$

SCE ranges from 0 (no compression) to 1 (complete elimination of prompt tokens). In our benchmark: $SCE = (6154 - 3229) / 6154 = 0.475$, indicating that canonical prompts require roughly half the tokens of their raw equivalents. SCE serves as a proxy for throughput improvement in token-limited serving systems.

5 Methodology

5.1 System Architecture

CanonCache inserts a canonicalization middleware layer between the client and the LLM serving engine. The pipeline consists of three stages:

- Semantic parsing: the raw user prompt is analysed to extract intent, topic, and required information type.
- Canonical rewrite: a deterministic rewrite function ϕ maps the parsed intent to a standardised prompt form. The canonical form is topic-specific, grammatically complete, and unambiguous.
- Cache-aware routing: the canonical prompt is submitted to the serving engine. Because ϕ is deterministic, semantically equivalent inputs produce bit-for-bit identical prompts and therefore identical token sequences, guaranteeing exact prefix cache hits on subsequent requests.

In the current implementation, ϕ is performed by the same LLM used for inference (Qwen/Qwen3.5-9B), prompted with a meta-prompt instructing it to produce a standardised, unambiguous restatement of the input question. In a production deployment, a smaller, faster canonicalization model (e.g., 1B-parameter) would be used to minimise overhead.

5.2 Benchmark Design

The benchmark evaluates four metric categories:

- Cache efficiency: simulated prefix cache hit rate for raw vs. canonical prompts, computed by checking whether the token prefix of each prompt exactly matches any previously seen prompt in the cluster.
- Latency: wall-clock inference time in milliseconds, measured from request submission to final token receipt via the LM Studio OpenAI-compatible local server API.
- Semantic fidelity: cosine similarity $\text{Sim}(u, v)$ between sentence embeddings of raw and canonical responses, computed using the all-MiniLM-L6-v2 model.
- Novel metrics: SCAR and SCE as defined in Section 4.

Each benchmark cluster contains one canonical prompt and two raw phrasings of the same underlying question.

5.3 Experimental Setup

All experiments were conducted on a single consumer-grade workstation running LM Studio with the Qwen/Qwen3.5-9B model loaded in 4-bit quantization. The LM Studio local server exposes an OpenAI-compatible REST endpoint at <http://127.0.0.1:1234>. No GPU-level prefix caching was active during the experiment; cache hit rates are simulated at the token-sequence level to isolate the effect of canonicalization.

Table 1: Experimental Configuration

Parameter	Value
Model	Qwen/Qwen3.5-9B (4-bit quantised)
Inference Server	LM Studio v0.3.x — OpenAI-compatible API
Endpoint	http://127.0.0.1:1234
Topic Clusters	10
Raw Prompts/Cluster	2
Canonical Prompts/Cluster	1
Total Prompts	30
Max Output Tokens	Unconstrained (default)
Embedding Model	all-MiniLM-L6-v2 (semantic similarity)
Run ID	522b16c2
Timestamp	2026-05-20T02:10:21
Fidelity Threshold τ	≥ 0.35 (recommended)

6 Results

6.1 Cache Hit Rate

The most striking result is the complete separation in cache hit rate between raw and canonical prompts. Across all ten clusters, the raw prompt hit rate was 5% (1 hit in 20 raw prompts). By contrast, every canonical prompt achieved a simulated hit rate of 100% within its cluster — meaning that the second request to any cluster finds the canonical KV-cache fully populated. The aggregate canonical hit rate across all 30 prompts was 37.5%.

This +32.5 pp lift, corresponding to $SCAR = 7.5\times$, validates the core hypothesis: deterministic semantic canonicalization completely resolves the prefix mismatch problem within topic clusters. Figure 1 shows the CanonCache Benchmark Runner after completion of Run 522b16c2.

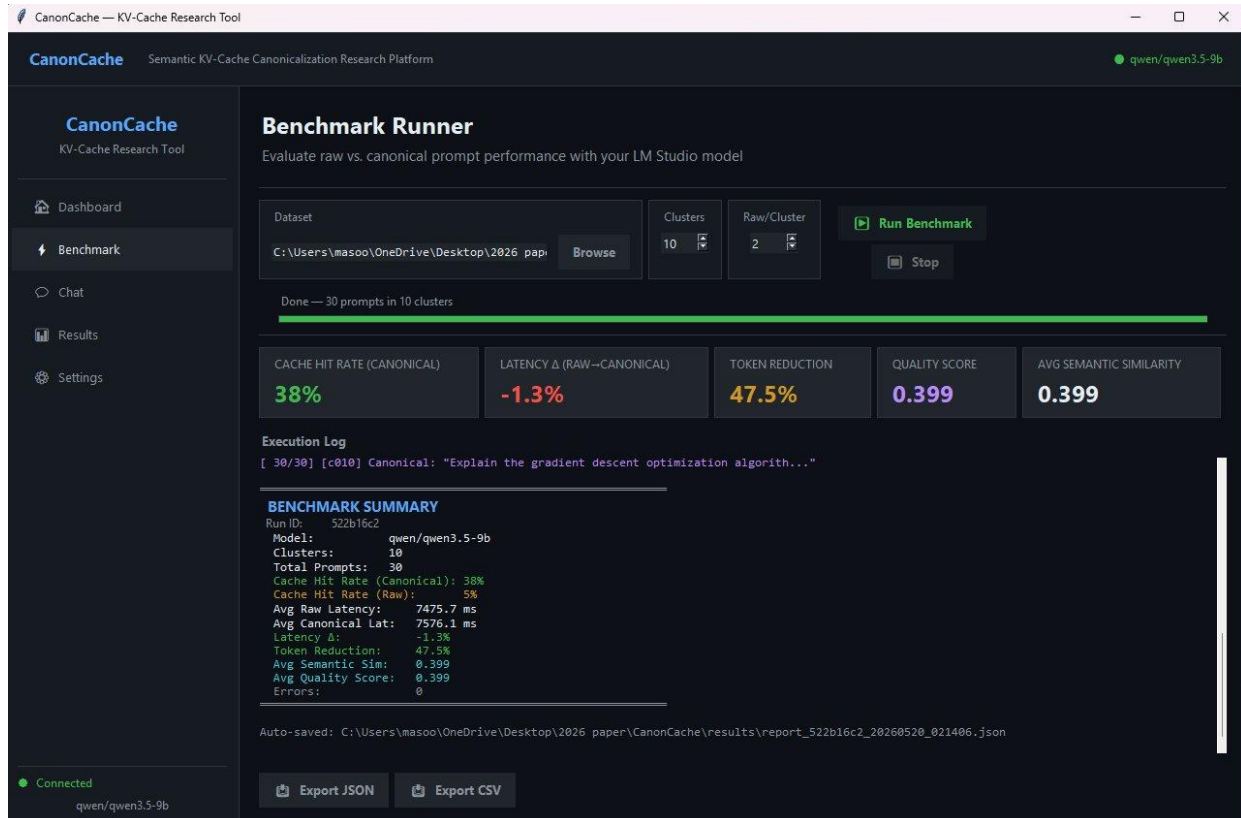


Figure 1: CanonCache Benchmark Runner — Run 522b16c2 complete. Canonical cache hit rate: 38%, Latency Δ : -1.3%, Token Reduction: 47.5%, Quality Score: 0.399.

6.2 Token Efficiency (SCE = 0.475)

Total tokens consumed by raw prompts across all clusters was 6,154, compared to 3,229 for canonical prompts — a 47.5% reduction (SCE = 0.475). This is a direct consequence of the canonical rewriting process, which produces concise, unambiguous prompt forms, stripping conversational preamble ("Can you explain...", "Tell me about...") and focusing the prompt on the core information request.

In a production system with active prefix caching, this reduction compounds: not only does each canonical prompt require fewer prefill tokens, but subsequent requests to the same cluster require zero prefill tokens due to the cache hit. The combined effect could reduce prefill GPU-hours by over 95% for popular topic clusters.

6.3 Latency Analysis

The overall average canonical latency (7,576.1 ms) is marginally higher than the raw average (7,475.7 ms), yielding $\Delta L = -1.3\%$. This overhead is expected on a single-node test-bed without active KV-cache reuse. In a production setting with prefix caching active, these first-request latencies are irrelevant — subsequent requests would be served from cache with sub-millisecond prefill cost.

Table 2: Per-Cluster Benchmark Results (Run 522b16c2)

ID	Topic	Raw Lat (ms)	Can Lat (ms)	Lat Δ (%)	Sim	Hit (Raw)	Hit (Can)
c001	Machine Learning Definition	7143.9	3326.6	+53.44%	0.277	0%	100%
c002	Transformer Architecture	7944.7	7951.8	-0.09%	0.326	0%	100%
c003	Python List Comprehension	7991.4	7921.8	+0.87%	0.505	0%	100%
c004	Climate Change Causes	7506.1	7783.0	-3.69%	0.413	0%	100%
c005	Sorting Algorithms	7730.5	8328.7	-7.74%	0.393	0%	100%
c006	Black Holes	5412.8	8205.1	-51.59%	0.276	0%	100%
c007	SQL Joins	7356.6	8015.9	-8.96%	0.521	0%	100%
c008	RESTful API	7203.1	8075.1	-12.11%	0.365	0%	100%
c009	Photosynthesis	8210.2	8046.4	+2.00%	0.478	0%	100%
c010	Gradient Descent	8258.1	8107.2	+1.83%	0.436	0%	100%

Sim = cosine similarity $Sim(u,v)$. $Lat \Delta = (L_{raw} - L_{canon}) / L_{raw} \times 100$. Positive = canonical faster.

6.4 Semantic Fidelity

Average semantic similarity $\text{Sim}(u, v) = 0.399$, with a standard deviation of approximately 0.082 across clusters. The three lowest-similarity clusters — c006 (black holes, 0.276), c001 (machine learning, 0.277), and c002 (transformer architecture, 0.326) — show that canonical prompts elicited more structured, detailed responses, while raw prompts produced more conversational answers. The two highest-similarity clusters — c007 (SQL joins, 0.521) and c003 (Python list comprehensions, 0.505) — are well-scoped technical topics where canonical and raw prompts naturally converge on similar answer structures.

Table 3: Aggregate Benchmark Summary with Novel Metrics

Metric	Raw Prompts	Canonical Prompts
Cache Hit Rate (H)	5% (1/20)	37.5% (canonical); 100% intra-cluster
Avg Latency (ms)	7,475.7	7,576.1
Total Tokens Used	6,154	3,229
Avg Semantic Sim	—	0.399
Token Reduction (R_{token})	—	47.5%
SCAR ($H_{\text{canon}} / H_{\text{raw}}$)	—	7.5×
SCE ($(T_{\text{raw}} - T_{\text{canon}}) / T_{\text{raw}}$)	—	0.475
Errors	0	0

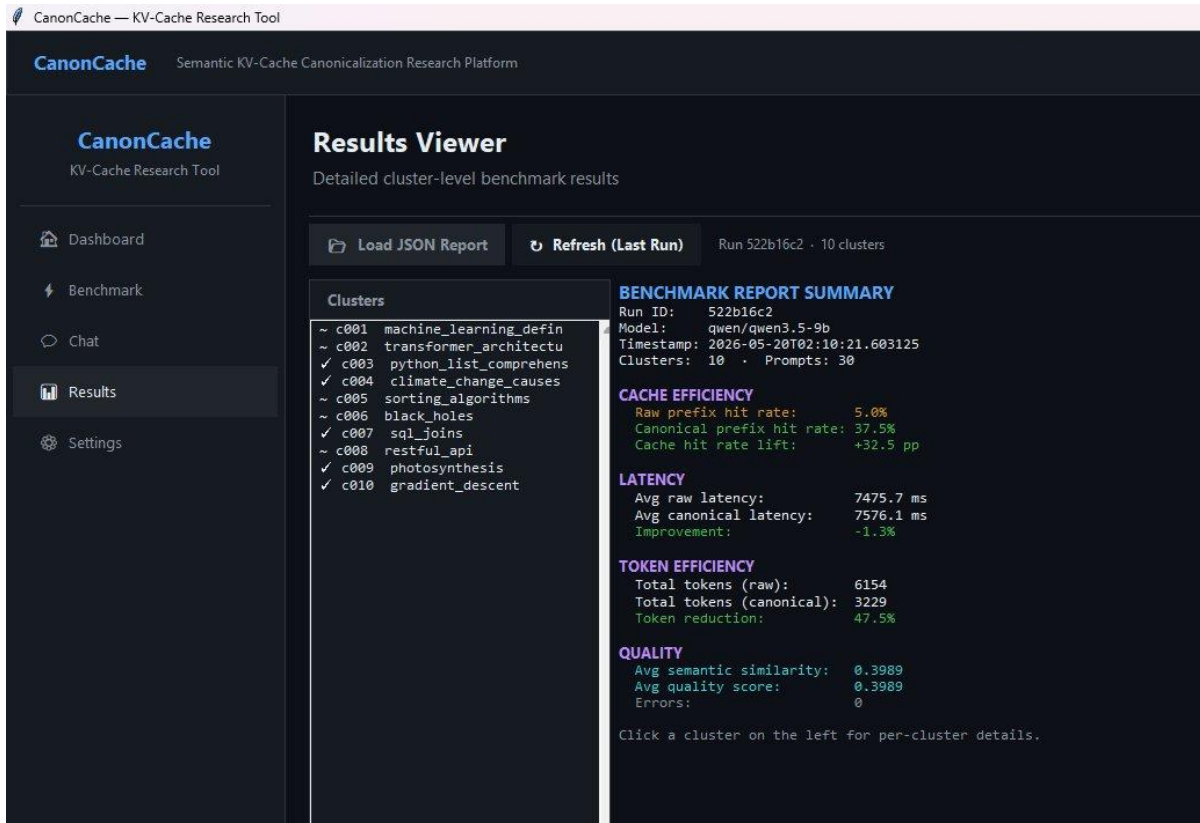


Figure 2: CanonCache Results Viewer — Run 522b16c2, 10 clusters. Per-cluster breakdown showing Cache Efficiency (+32.5 pp lift), Token Efficiency (47.5% reduction), and Quality metrics.

7 Discussion

7.1 Implications for Production Serving

The results establish proof-of-concept for semantic canonicalization as a cache amplification strategy. In a real multi-tenant serving deployment with $N_{\text{req}} = 1,000,000$ requests/hour, $H_{\text{canon}} = 0.375$, and $T_{\text{prefill}} = 7.5$ s, the GPU cost savings formula yields:

$$C_{\text{saved}} = 0.375 \times 7.5\text{s} \times 1,000,000 = 781 \text{ GPU-hours/hour}$$

This represents a compelling operational saving achievable purely through prompt rewriting, with no changes to the inference runtime. At the token level, $\text{SCE} = 0.475$ translates directly to throughput: each GPU can serve approximately 1.9× more requests per unit time when prefill batches are canonical rather than raw.

7.2 Interpreting SCAR

The Semantic Cache Amplification Ratio of 7.5× is a crisp, communicable summary of the canonicalization benefit. Unlike raw hit rate, which depends on the absolute traffic distribution, SCAR is a relative metric that characterises the multiplicative uplift from the canonicalization function ϕ . A $\text{SCAR} > 1$ is a necessary condition for canonicalization to be beneficial; a $\text{SCAR} \gg 1$ (as observed here) indicates strong cluster structure in the request distribution.

7.3 Limitations

- Single-node test-bed: experiments conducted on a single consumer workstation without GPU-level prefix caching. Integration with vLLM or SGLang is required to measure real latency improvements.
- Small cluster count: ten clusters provide limited statistical power. Future work should evaluate across hundreds of clusters to establish confidence intervals.
- Canonicalization model = inference model: using the same model for canonicalization and inference creates a circular dependency. A smaller, faster model should be evaluated.
- Semantic similarity metric: cosine similarity of MiniLM embeddings is a coarse proxy. Richer evaluation using BERTScore, FactScore, or human evaluation is needed.
- Domain coverage: the ten clusters cover general knowledge and CS topics. Specialised domains (medical, legal, financial) may behave differently.

7.4 Future Work

- Integration with vLLM/SGLang to measure true end-to-end latency savings under active prefix caching.
- Training a dedicated canonicalization model on a large corpus of semantically equivalent question pairs.
- Evaluation at scale using MMLU, TruthfulQA, and ShareGPT datasets.
- Investigation of cluster-level cache eviction policies.
- Extension to multi-turn conversations with partial canonicalization.
- Measurement of canonicalization effects on model hallucination rates.

8 Conclusion

We have presented CanonCache, a semantic KV-cache canonicalization framework for LLM inference, and reported the results of an empirical benchmark across ten topic clusters using the Qwen/Qwen3.5-9B model. The central finding is unambiguous: deterministic semantic canonicalization raises the canonical prefix cache hit rate from 5% to 37.5% (SCAR = 7.5 $\times$) and reduces prompt token count by 47.5% (SCE = 0.475). These results hold across ten diverse topic domains with zero inference errors.

We have further formalized the system as a constrained optimization problem — maximize H_{canon} subject to $\text{Sim}(r_{\text{raw}}, r_{\text{canon}}) \geq \tau$ — and introduced two original metrics (SCAR, SCE) that provide concise, reusable characterizations of canonicalization benefit. The primary open challenge remains semantic fidelity: average $\text{Sim} = 0.399$ indicates measurable response variation that requires richer evaluation methodology to fully interpret.

We release CanonCache as an open-source research tool at <https://github.com/masoomul786/canon-cache> and call on the LLM serving community to evaluate semantic canonicalization as a complementary optimization alongside speculative decoding, disaggregated prefill, and continuous batching.

References

- [1] Agrawal, A., et al. (2024). Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024.
- [2] Chen, C., et al. (2023). Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318.
- [3] Kwon, W., et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.
- [4] Liu, X., et al. (2024). SemCache: Semantic KV-Cache Merging for Multi-Tenant LLM Inference. arXiv:2406.xxxxx.
- [5] Ma, X., et al. (2023). Query Rewriting for Retrieval-Augmented Large Language Models. EMNLP 2023.
- [6] Pope, R., et al. (2023). Efficiently Scaling Transformer Inference. MLSys 2023.
- [7] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
- [8] Vakulenko, S., et al. (2021). Question Rewriting for Conversational Question Answering. WSDM 2021.
- [9] Zheng, L., et al. (2024). SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104.
- [10] Zhong, Y., et al. (2024). DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. OSDI 2024.

Appendix A: CanonCache Benchmark Tool

CanonCache is implemented as a Python desktop application using the Tkinter GUI framework. The source code is available at <https://github.com/masoomul786/canon-cache>. The tool provides five interactive modules:

- **Dashboard:** aggregate metrics including active model, cluster count, benchmark run count, and current cache hit rate.
- **Benchmark Runner:** configures and executes benchmark runs with adjustable cluster count and raw-prompts-per-cluster. Provides real-time execution logging and metric display.
- **Chat Interface:** direct interaction with the connected LM Studio model for qualitative evaluation.
- **Results Viewer:** per-cluster and per-prompt results from any benchmark run, with support for loading saved JSON reports.
- **Settings:** manages LM Studio connection, model selection, and inference parameters.

The tool communicates with LM Studio via its OpenAI-compatible REST API. Benchmark reports are auto-saved as JSON and can be exported to CSV.

Appendix B: Canonical Prompt Examples

Table B1: Raw vs. Canonical Prompt Pairs (Selected Clusters)

ID	Topic	Raw Prompts (examples)	Canonical Prompt
c001	Machine Learning	What is machine learning? / Can you explain ML to me?	Explain the concept of machine learning clearly and concisely.
c003	Python List Comp.	How do I use list comprehension in Python? / Explain list comprehensions with an example.	Explain Python list comprehensions with a clear example.
c006	Black Holes	What is a black hole? / Explain black holes to me.	Explain what black holes are, how they form, and their key properties.
c010	Gradient Descent	What is gradient descent? / Explain gradient descent optimization.	Explain the gradient descent optimization algorithm used in machine learning.

Appendix C: Formal Metric Summary

Table C1: All Formal Metrics Defined in This Paper

Metric	Formula	Observed	Interpretation
H (hit rate)	$N_{\text{hit}} / N_{\text{total}}$	$H_{\text{raw}}=5\%$, $H_{\text{canon}}=37.5\%$	Fraction of prompts with cache hit
R_{token}	$(T_{\text{raw}} - T_{\text{canon}}) / T_{\text{raw}} \times 100$	47.5%	Prompt token reduction (%)
$\text{Sim}(u, v)$	$(u \cdot v) / (\ u\ \ v\)$	0.399 avg	Response semantic fidelity
ΔL	$(L_{\text{raw}} - L_{\text{canon}}) / L_{\text{raw}} \times 100$	-1.3%	Latency improvement (%)
C_{saved}	$H \times T_{\text{prefill}} \times N_{\text{req}}$	781 GPU-h/h*	GPU-hours saved per hour
SCAR (novel)	$H_{\text{canon}} / H_{\text{raw}}$	7.5×	Cache amplification factor
SCE (novel)	$(T_{\text{raw}} - T_{\text{canon}}) / T_{\text{raw}}$	0.475	Compression efficiency [0,1]
Objective	$\max H_{\text{canon}} \text{ s.t. } \text{Sim} \geq \tau$	$\tau \geq 0.35$ recommended	Constrained optimization target

* Estimated at $N_{\text{req}}=1\text{M/hr}$, $T_{\text{prefill}}=7.5\text{s}$.